Compréhension effort de développement pour plugin Canva *Face Blurring*

But : plugin de *Face Blurring* pour Canva :

Entrées du plugin : image ou vidéo à importer, choix du type de floutage (*check*, *blur* ou *pixelate*), paramètres de floutage (*strength*, *feather* et *padding*)

Sorties du plugin : image ou vidéo avec les visages floutés à intégrer dans le design

Le développement d'un plugin pour Canva nécessite plusieurs étapes clés pour s'assurer que le plugin est efficace, fonctionnel et en conformité avec les exigences de la plateforme. Voici un aperçu des étapes essentielles :

1. Recherche et Planification

- **Analyse des besoins** : Comprendre ce que votre plugin doit accomplir et définir clairement ses fonctionnalités.
- **Étude de marché** : Examiner les plugins existants pour identifier les opportunités et les fonctionnalités manquantes.
- **Définir l'audience cible** : Identifier qui utilisera le plugin et comment il s'intégrera dans leurs flux de travail.

2. Design et Prototypage

- **Wireframing et Mockups** : Créer des maquettes et des wireframes pour visualiser l'interface utilisateur et l'expérience utilisateur (UX).
- **Prototype** : Développer un prototype interactif pour tester l'ergonomie et l'interactivité.

3. Développement

- **Choix des technologies** : Sélectionner les technologies et les outils appropriés (par ex., langage de programmation, frameworks).
- **Architecture du plugin** : Concevoir l'architecture logicielle du plugin.
- **Développement Front-end** : Créer l'interface utilisateur du plugin en utilisant HTML, CSS, et JavaScript.
- **Développement Back-end** : Mettre en place le serveur et les bases de données nécessaires pour les fonctionnalités du plugin (si nécessaire).
- **Intégration avec Canva** : Utiliser l'API de Canva pour intégrer votre plugin à la plateforme.

4. Test et Validation

- **Tests unitaires** : Vérifier chaque composant du plugin individuellement.
- **Tests d'intégration** : Tester l'interaction entre différents composants du plugin et entre le plugin et Canva.
- **Tests utilisateur** : Obtenir des retours d'utilisateurs réels pour améliorer l'expérience utilisateur et corriger les bugs.

5. Documentation

- **Documentation technique** : Rédiger une documentation détaillée pour les développeurs sur l'installation, l'utilisation et la maintenance du plugin.
- **Guides utilisateur** : Créer des guides et des tutoriels pour aider les utilisateurs à comprendre

et à utiliser le plugin efficacement.

6. Déploiement

- **Préparation pour le lancement** : S'assurer que le plugin est prêt pour le déploiement, y compris la correction des bugs et l'optimisation des performances.
- **Publication** : Soumettre le plugin à Canva pour examen et approbation, puis le publier sur la plateforme.

7. Maintenance et Support

- **Mise à jour et amélioration** : Continuer à améliorer le plugin en ajoutant de nouvelles fonctionnalités et en corrigeant les problèmes.
- **Support utilisateur** : Offrir un support technique et répondre aux questions des utilisateurs.

8. Marketing et Promotion

- **Stratégie de lancement** : Planifier et exécuter une stratégie de marketing pour promouvoir le plugin.
- **Engagement communautaire** : Interagir avec la communauté Canva pour recueillir des retours et accroître l'adoption du plugin.

En suivant ces étapes, vous pouvez vous assurer que le développement de votre plugin pour Canva se déroule de manière structurée et efficace, conduisant à un produit final de haute qualité qui répond aux besoins de vos utilisateurs.

Points critiques de développement à prendre en compte :

- **authentification** des utilisateurs et lien avec la base de données côté serveur Spooqs
- **communication**, transfert des données nécessaires sur le serveur Spooqs
- mise en place d'un **traitement** sur le serveur pour gérer les demandes
- envoi des données traitées et récupération par le plugin

Getting started
OverviewPrerequisitesQuick startAPI reference
Designing apps
App UI Kit
Design guidelines
Figma resource
Fundamentals
Creating appsSetting up the starter kitPreviewing appsIntegrating with CanvaConfiguring permissionsHandling errorsUsing a backendBundling apps
Developing apps
Authenticating users
Creating app elementsCreating audio tracksCreating imagesCreating image overlaysCreating shapesCreating tablesCreating textCreating videosEmbedding rich mediaExporting designsGrouping elementsPositioning elementsReading elementsReplacing elementsSupporting drag and dropUploading assetsUsing color selectorsUsing design IDs
Resources
App components
App templates
Securing apps
Content Security PolicyCross-Origin Resource SharingVerifying JWTsVerifying HTTP

requestsSecurity guidelines
Monetizing apps
Innovation FundAccepting payments
Distributing apps
App review processSubmitting appsSubmission checklistApp listing guidelines
Releasing appsTracking analyticsVersioning appsHandling disabled appsDeep linking to appsManaging team appsGetting featuredDeveloper verification

How to create a plugin for Canva?

1. Prerequisites.
2. Step 1: Set up a Base URL.
3. Step 2: Create an app via the Developer Portal.
4. Step 3: Configure the content extension.
5. Step 4: Preview the extension in the Canva editor.
6. Next steps. Example.

Can ChatGPT make Canva?

You can now plugin Canva to your ChatGPT account. Use your imagination to describe the Canva template that you want to generate using ChatGPT.

Is Canva API free?

Yes, anyone can use the Connect APIs free of charge to build integrations that will bring Canva functionality into another platform, system, or tool. How do I get started with the Canva Connect APIs?

What programming language does Canva use?

The app itself is written in **TypeScript & React**, using the Canva SDK. What's neat is that it can connect to your own backend, and guess what? You can write that backend in Python! So, even though we're talking about Canva App Development, there's still a little bit of Python involved.

How do I create a custom plugin?

How to create a custom plugin:

1. Step 1 – Scaffold the plugin files. The following command uses several options to lets us specify the plugin slug, its name, description, author name and uri as well as the plugin uri. ...
2. Step 2 – create a custom post type: ...
3. Step 3 – Write code inside the main file: ...
4. Step 4 – Activate the plugin.

Canva Developers Documentation page : https://www.canva.dev/docs/apps/

Embed Canva content in Wordpress : https://embedpress.com/blog/embed-canva-content-in-wordpress/

Canva apps sdk starter kit github : https://github.com/canva-sdks/canva-apps-sdk-starter-kit

Documentation de user interface kit : https://www.canva.dev/docs/apps/app-ui-kit/storybook/?path=/docs/canva-app-ui-kit-action-button--docs

Quick start

Get an app up and running in a matter of minutes.

Canva's app development platform allows anyone to create apps that enhance Canva. This quick start guide explains how to get an app up and running in a matter of minutes.

Before you begin

To follow this guide, you'll need to:

- Sign up for a Canva.com account.
- Install git, Node.js v20.10.0, and npm v10.
- Learn the fundamentals of TypeScript, React, and webpack.

If you need help with any of these prerequisites, including how to set up the required tooling, see Prerequisites.

Step 1: Create an app via the Developer Portal

1. Log in to the Developer Portal.
2. Navigate to the Your apps page.
3. Click **Create an app**.
4. Select a target audience for the app:
    - **Public**: You can make your app available to all of Canva's users, but the app will need to be reviewed by Canva and meet the requirements outlined in the submission checklist.
    - **Restrict to your team**: The app can only be made available to members of the current team, and the team's administrators are responsible for reviewing it.
5. Agree to the terms and conditions.
6. Click **Create**.

Step 2: Set up the starter kit

1. Clone the following repo:

    ```
    git clone https://github.com/canva-sdks/canva-apps-sdk-starter-kit.git
    ```

    bash

- This repo contains the *starter kit* for the Apps SDK. The starter kit is a boilerplate for an app and all of the tooling we recommend. To learn more about the starter kit, see Setting up the starter kit.
- Navigate into the cloned directory for the starter kit:

```
cd canva-apps-sdk-starter-kit
```

bash

- Install the dependencies:

```
npm install
```

bash

Start the local development server:

```
npm run start
```

bash

4. The local development server starts running at http://localhost:8080, but you can't use this URL to view the local preview, see the next step instead.

Step 3: Preview the app

This step explains how to use Canva to view the app running locally at http://localhost:8080.

1. In the Developer Portal, navigate to the app's **Configure your app** page.
2. Select **App source > Development URL**.
3. In the **Development URL** field, enter the URL of the development server.
4. To open the app in a new tab, click **Preview**.
5. To install the app, click **Open**.

The example app includes a button that responds with "Hello world" when clicked:

...

To learn more about how to preview apps, including how to set up live reloading, see Previewing apps.

Step 4: Start editing code

You can quickly get started by editing the example app demonstrated above.

Use a code editor to open `src/app.tsx`. This file contains placeholder code that you can edit to start building your app.

In the following example, a button click triggers the addNativeElement method, adding a new text layer to the Canva design with the words "Hello world!".

```
import { Button, Rows, Text } from "@canva/app-ui-kit";
import { addNativeElement } from "@canva/design";
import * as React from "react";
import styles from "styles/components.css";

export const App = () => {
  const onClick = () => {
    addNativeElement({
      type: "TEXT",
      children: ["Hello world!"],
    });
  };

  return (
    <div className={styles.scrollContainer}>
```

```
    <Rows spacing="2u">
      <Text>
        To make changes to this app, edit the{" "}
        <code className={styles.code}>src/app.tsx</code> file, then close and
        reopen the app in the editor to preview the changes.
      </Text>
      <Button variant="primary" onClick={onClick} stretch>
        Do something cool
      </Button>
    </Rows>
  </div>
 );
};
```

Next steps

- You can use various Canva APIs to add functionality to your app. For more information, see Integrating with Canva.
- If you plan to submit your app to the Apps Marketplace, you can prepare by reading the following:
    - Design guidelines
    - App listing guidelines
    - App review process
    - Submission checklist
    - Submitting apps

Canva developer terms : https://www.canva.com/policies/canva-developer-terms/

Canva marketplace : https://www.canva.com/apps/

Canva security guidelines : https://www.canva.dev/docs/apps/security-guidelines/

Reasonable security measures : https://owasp.org/www-project-top-ten/#

Canva developers community : https://community.canva.dev/

First steps

- To start making an app, see Quick start.
- To explore example apps, check out the starter kit.
- If you have any questions, create a support ticket.

In the iframe, apps can:

- Use the Apps SDK to programmatically interact with the Canva editor.
- Use the App UI Kit to create a user interface that matches the look and feel of Canva.
- Use the Fetch API to integrate with the app's backend.

TL;DR (Too long; didn't read) Section Prerequisites for more details

You'll need:

- A relatively modern web browser
- A free Canva.com account
- A command line application, such as Terminal on macOS
- An installation of git, Node.js (v20.10.0), and npm (v10)
- Some knowledge of TypeScript, React, and webpack

Quick start : To follow this guide, you'll need to:

- Sign up for a Canva.com account.
- Install git, Node.js v20.10.0, and npm v10.
- Learn the fundamentals of TypeScript, React, and webpack.

install git avec brew install git

spooqs@MacBook-Pro-de-Spooqs ~ % curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.5/install.sh | bash

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 15916  100 15916    0     0   223k      0 --:--:-- --:--:-- --:--:--  225k
=> Downloading nvm from git to '/Users/spooqs/.nvm'
=> Clonage dans '/Users/spooqs/.nvm'...
remote: Enumerating objects: 369, done.
remote: Counting objects: 100% (369/369), done.
remote: Compressing objects: 100% (315/315), done.
remote: Total 369 (delta 42), reused 166 (delta 27), pack-reused 0
Réception d'objets: 100% (369/369), 368.22 Kio | 5.11 Mio/s, fait.
Résolution des deltas: 100% (42/42), fait.
* (HEAD détachée sur FETCH_HEAD)
```

```
  master
=> Compressing and cleaning up git repository

=> Profile not found. Tried ~/.bashrc, ~/.bash_profile, ~/.zprofile, ~/.zshrc, and ~/.profile.
=> Create one of them and run this script again
   OR
=> Append the following lines to the correct file yourself:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  # This loads nvm

=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  # This loads nvm
spooqs@MacBook-Pro-de-Spooqs ~ %
```

Installing Node.js

To install Node.js, we recommend using Node Version Manager (nvm). This is a tool for managing multiple versions of Node.js on a single system, which reduces the risk of frustrating versioning errors.

- Install nvm by running the following command:
  curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.5/install.sh | bash

  export NVM_DIR="$HOME/.nvm"

[-s "$NVM_DIR/nvm.sh"] && \. "$NVM_DIR/nvm.sh" # This loads nvm

- TODO : **At the root of the starter repo, install Node.js by running the following command: nvm install**

To verify that the correct version is available in the current terminal session, run the following command:
node -v

If the correct version is not returned, run the following command:
nvm use 20.10.0

Or, if you're in the directory for the starter kit, simply run:
nvm use

This will set the correct version based on the starter kit's .nvmrc file.

Attention : à faire avant -> touch ~/.zshrc , exécuter la commande curl ci-dessus, puis fermer et réouvrir le terminal, aller dans le répertoire canva-apps-sdk-starter-kit et faire l'installe de nvm

```
Last login: Thu Jul 11 07:57:44 on ttys000
spooqs@MacBook-Pro-de-Spooqs ~ % cd canva-apps-sdk-starter-kit
spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % nvm install
Found '/Users/spooqs/canva-apps-sdk-starter-kit/.nvmrc' with version <20.10.0>
Downloading and installing node v20.10.0...
Downloading https://nodejs.org/dist/v20.10.0/node-v20.10.0-darwin-x64.tar.xz...
#################################################################### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v20.10.0 (npm v10.2.3)
Creating default alias: default -> 20.10.0 (-> v20.10.0)
spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % node -v
v20.10.0
spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % npm install npm@10 --global
```

removed 41 packages, and changed 87 packages in 3s

22 packages are looking for funding
  run `npm fund` for details
spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % npm fund
canva-apps-sdk-starter-kit

spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % npm install
npm warn deprecated popper.js@1.16.1: You can find the new Popper v2 at @popperjs/core, this package is dedicated to the legacy v1

added 969 packages, and audited 1010 packages in 14s

109 packages are looking for funding
  run `npm fund` for details

found **0** vulnerabilities
spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit %

Step 1: Create an app via the Developer Portal

- Log in to the Developer Portal.
- Navigate to the Your apps page.
- Click Create an app.
- Select a target audience for the app:
  - Public: You can make your app available to all of Canva's users, but the app will need to be reviewed by Canva and meet the requirements outlined in the submission checklist.
  - Restrict to your team: The app can only be made available to members of the current team, and the team's administrators are responsible for reviewing it.
- Agree to the terms and conditions.
- Click Create.

Step 2: Set up the starter kit

- Clone the following repo:
-

  This repo contains the starter kit for the Apps SDK. The starter kit is a boilerplate for an app and all of the tooling we recommend. To learn more about the starter kit, see Setting up the starter kit.
- Navigate into the cloned directory for the starter kit:
  cd canva-apps-sdk-starter-kit
- Install the dependencies:
  npm install
- Start the local development server:
  npm run start

  The local development server starts running at http://localhost:8080, but you can't use this URL to view the local preview, see the next step instead.

  spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % npm run start

> start
> ts-node ./scripts/start/start.ts

Info: Starting development server for **/Users/spooqs/canva-apps-sdk-starter-kit/src**

Note: Hot Module Replacement (HMR) not enabled. To enable it, please refer to the instructions in the **README.md**

Development URL (Frontend)	http://localhost:8080

Note: For instructions on how to set up the app via the Developer Portal, see the **README.md**.

asset **app.js** 4.8 MiB **[emitted]** (name: app) 1 related asset
900 modules
webpack 5.90.3 compiled **successfully** in 4806 ms

App Origin : https://app-aagknrub2km.canva-apps.com

Using HTTPS
By default, the development server does not use the HTTPS protocol. This is convenient because there's no need for a security certificate, but it prevents you from previewing an app in Safari.
If you're using Safari (or otherwise need to use HTTPS):

- Start the development server with the --use-https flag:
  npm run start --use-https
- Navigate to https://localhost:8080.
- Bypass the invalid security certificate warning:
  - Click Show details.

Click Visit website.

spooqs@MacBook-Pro-de-Spooqs canva-apps-sdk-starter-kit % npm run start --use-https

> start
> ts-node ./scripts/start/start.ts

Info: Starting development server for **/Users/spooqs/canva-apps-sdk-starter-kit/src**

Note: Hot Module Replacement (HMR) not enabled. To enable it, please refer to the instructions in the **README.md**

Development URL (Frontend)	https://localhost:8080

Note: For instructions on how to set up the app via the Developer Portal, see the **README.md**.

asset **app.js** 4.8 MiB **[emitted]** (name: app) 1 related asset
900 modules
webpack 5.90.3 compiled **successfully** in 4797 ms

Step 3: Preview the app

This step explains how to use Canva to view the app running locally at http://localhost:8080.

- In the Developer Portal, navigate to the app's Configure your app page.
- Select App source > Development URL.
- In the Development URL field, enter the URL of the development server.
- To open the app in a new tab, click Preview.
- To install the app, click Open.

The example app includes a button that responds with "Hello world" when clicked:
To learn more about how to preview apps, including how to set up live reloading, see Previewing apps.

Sous Windows :
install git

```
C:\Users\vgiro>"C:/Program Files/WindowsApps/Microsoft.DesktopAppInstaller_1.23.1911.0_x64__8wekyb3d8bbwe/winget.exe" install -e --id CoreyButler.NVMforWindows
La source 'msstore' nécessite que vous consultiez les contrats suivants avant de l'utiliser.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
La source nécessite que la région géographique à 2 lettres de l'ordinateur actuel soit envoyée au service principal pour fonctionner correctement (par exemple, « ÉTATS-UNIS »).

Acceptez-vous toutes les conditions des contrats sources ?
[Y] Oui  [N] Non: Y
Trouvé NVM for Windows [CoreyButler.NVMforWindows] Version 1.1.12
La licence d'utilisation de cette application vous est octroyée par son propriétaire.
Microsoft n'est pas responsable des paquets tiers et n'accorde pas de licences à ceux-ci.
Téléchargement en cours https://github.com/coreybutler/nvm-windows/releases/download/1.1.12/nvm-setup.exe
[redacted] 5.51 MB / 5.51 MB
Le code de hachage de l'installation a été vérifié avec succès
Démarrage du package d'installation... Merci de patienter.
Installé correctement

C:\Users\vgiro>git clone https://github.com/canva-sdks/canva-apps-sdk-starter-kit.git
Cloning into 'canva-apps-sdk-starter-kit'...
remote: Enumerating objects: 1194, done.
remote: Counting objects: 100% (344/344), done.
remote: Compressing objects: 100% (213/213), done.
remote: Total 1194 (delta 206), reused 219 (delta 128), pack-reused 850
Receiving objects: 100% (1194/1194), 1009.44 KiB | 8.70 MiB/s, done.
Resolving deltas: 100% (506/506), done.

C:\Users\vgiro\canva-apps-sdk-starter-kit>nvm install 20.10.0
Downloading node.js version 20.10.0 (64-bit)...
Extracting node and npm...
Complete
npm v10.2.3 installed successfully.


Installation complete. If you want to use this version, type

nvm use 20.10.0

C:\Users\vgiro\canva-apps-sdk-starter-kit>nvm use 20.10.0
Now using node v20.10.0 (64-bit)

C:\Users\vgiro\canva-apps-sdk-starter-kit>

C:\Users\vgiro\canva-apps-sdk-starter-kit>npm install
npm WARN deprecated popper.js@1.16.1: You can find the new Popper v2 at @popperjs/core, this package is dedicated to the legacy v1
```

```
added 967 packages, and audited 1008 packages in 10s

109 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New minor version of npm available! 10.2.3 -> 10.8.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.8.2
npm notice Run npm install -g npm@10.8.2 to update!
npm notice

C:\Users\vgiro\canva-apps-sdk-starter-kit>npm install npm@10 --global

removed 41 packages, and changed 87 packages in 2s

C:\Users\vgiro\canva-apps-sdk-starter-kit>npm install @canva/app-ui-kit

up to date, audited 1008 packages in 2s

109 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

NOTE IMPORTANTE : sous mozilla, aller à l'adresse https://localhost:8080/ et bypasser l'alerte concernant le certificat de sécurité pour accéder au code et installer l'appli.

AUTHENTICATION :

Authentication
Guidelines for authenticating users.

You can configure apps to remember user data, whether by using the `auth.getCanvaUserToken` API or connecting with a third-party platform. This functionality is typically used to make certain features and content available only to users who authenticate, or to only a subset of users. This page contains some guidelines for creating a delightful authentication flow.

Authentication in your app's workflow

Canva is a visual design platform with users who want to experience an app and quickly and easily see its value.

Manual authentication (creating a login pop-up) can be a way to place users' content behind a gate, so it could be considered helpful to implement it as part of your app's workflow. However, requiring a user to log in before they can see as much function as possible can be enough to deter them from continuing to use your app.

There are many variations of authentication workflows, but some examples can produce a better experience for your users and, in turn, can lead to more usage of your app. For example, if your app requires the user to log in just so that you can see data about who is using your app, consider using user tokens instead of a login requirement.

Case study

A version of an app recently featured on Canva had some notable functions behind a paywall. This deterred users, with only 25% moving past the authentication stage. This app submitted a new version where a user had access to all the app's features over a 14-day trial period and quickly saw its daily users jump up 3750%. Now, the app has 123k installs and 3.8k uses monthly.

Authentication workflows

Where appropriate and relevant, we recommend you use the following list as an order of preference. The higher on this list your app falls, the better the user experience, which in turn translates to more users of your app.

1. All functions are available to the user. No authentication.
2. All functions are available to the user, regardless of whether the user authenticates.
3. Some functions of the app are available to the user, but they can still complete a basic workflow. The user must log in to use the other functions.
4. Some functions of the app are available to the user to test the app, but they cannot complete a workflow. The user must log in to complete the workflow.
5. No functions of the app are available to the user unless they authenticate.

You can use frictionless authentication for every option in the list above, and it should be considered over authentication when you want to track user data (time, usage), when you want to spot returning users, or when you want to implement a freemium model.

Authentication and the Apps Marketplace

An app can quickly and easily show its usefulness to a user by being promoted within the Apps Marketplace, such as inclusion in the "Featured apps" section.

However, an app is not eligible to be "Featured" unless a user can test functionality *before* manual authentication occurs in the workflow. This means that only apps that have no authentication or those that include a timed or per-use trial can be featured apps.

General guidelines

Authentication flow guidelines

- Where possible, make the app usable and useful before the user has to authenticate. Give them a chance to benefit from the app as quickly and as seamlessly as possible.
- Authentication flows must occur in the authentication pop-up window, not within the app's iframe — that is, sign-up and login forms should only appear in the pop-up window.
- Start an authentication flow by calling the `requestAuthentication` method. This will display a screen with a **Connect** button. Do not show redundant authentication screens before this screen or attempt to bypass the standard authentication flow.
- Prevent users from arriving at dead-ends or becoming stuck in endless loops.
- Support non-desktop devices, such as mobile phones and tablets.
- *Do not* auto-subscribe users to marketing content — all marketing material *must* be opt-in.
- If the same user installs the same app under different teams, require them to authenticate separately for each team — do not automatically authenticate the user.
- When a user disconnects (uninstalls) the app, delete the connection between the app and the third-party platform. If the user reconnects the app, they should be required to re-authenticate.

Pop-up guidelines

- For the pop-up page, use the `title` HTML element to set a meaningful title on the authentication's pop-up window.
- Provide clear and actionable error messages.
- Ensure that the redirect URL appears familiar and friendly — that is, not like a phishing threat.
- Provide options for both signing up for and logging in to the platform.

Suggested written content for authentication flows

There can be many scenarios to consider when building an authentication flow. You are free to make your own written content to communicate authentication flows. Alternatively, to speed up your development, we've provided some suggested written content that you can use.

Scenario	Suggested written content
Password is incorrect	Forgot password?
No email found	We couldn’t find that account. Try a different email or sign up.
One or all of the credentials are invalid	We couldn’t find that account. Try logging in a different way or sign up.
User needs to reset password	For your security, we’ve emailed your a link to reset your password.
Too many failed login attempts	Too many attempts. Please try again in X minute(s).
Password reset link expired or used	Looks like you need a new password reset link.

Scenario	Suggested written content
already	
User already has an account	Looks like you have an account already! Log in.
Not all form fields complete	Not quite done yet...
Incomplete fields	You missed this one
Passwords don’t match	Those passwords don’t match
Invalid password– doesn’t meet criteria	Use 8 or more characters with a mix of letters, numbers, and symbols.

Authenticating users

How to authenticate Canva users.

Authentication in Canva is about creating a seamless experience by ensuring that your app remembers your users. How you create this experience in your app can depend on the type of app and how you want to store user data.

Check out the Authentication design guidelines

Our design guidelines help you create a high-quality app that easily passes app review.

Types of authentication

Apps can handle users with two forms of authentication: frictionless, or manual.

Frictionless authentication (preferred)

Users confronted with a login prompt could consider it a hurdle. Any hurdles in the onboarding process come with a chance for them to drop out of your funnel, maybe to never return again. Because of this, we encourage you to consider *frictionless authentication*. This ensures that user data or credentials are automatically remembered to help users retain data, even without them passing through a login screen.

For more information on adding frictionless authentication to your app, see Frictionless authentication.

Manual authentication

This requires your users to authenticate with a third-party platform to access certain content or features within an app, including their own data. Your app can use manual authentication, but the way you handle users and user data can affect your apps inclusion in the user adoption program and your app's positioning in the Apps Marketplace. Consider using frictionless authentication, and make sure to follow our authentication guidelines.

For more information on adding manual authentication to your app, see Manual authentication.

Authentication workflows

How you create an app and create ways for users to authenticate can shape the user experience. The following are workflows you can consider to create a delightful authentication flow.

Handle existing or returning users

If a Canva user has an account with your app (for example, if they have previously accessed your app with Canva) fetching the user token should be enough for you to identify if they do. Then you can either:

- Continue allowing them access to the content associated with that user token.
- Prompt them to log in to their account using manual authentication.

A user might have an account with your app, but have never accessed your app through Canva, so you don't have a user token associated to them.

In this case we recommend adding non-blocking text such as 'Already have an account? Click here to sign in.' at the bottom of your app to provide them the opportunity to initiate the authentication process.

Support a free trial without authentication

If the trial is time-based, you can save the date of initial account creation and then check if that date + [trial time period] is before today's date. If it is, then allow the user to continue accessing the app. If it is not, you can restrict the UI in the app until the user authenticates and subscribes to an appropriate plan.

If the trial is usage-based, track usage as above and then restrict usage of the app when the tracked usage is the same as your defined usage limit.

Free and freemium

If your app is free, you can use tracking usage for your analytics to understand how users are using your app. Free apps typically see higher usage, which count towards your app usage award tier - so you can make money without ever needing to set up subscriptions or authenticating users!

If you implement a freemium model for your app, which allows a certain amount of free usage per month (same as free trial), get them to upgrade, but reset at the end of the month.

If they can only use certain features, consider:

- Allowing access to those features within the free tier
- When they want to try a paid feature, give them some access to trial it (and track as you would any other usage and then require users authenticate and then subscribe to continue using that feature).

API reference

- `auth.getCanvaUserToken`
- `auth.requestAuthentication`

Frictionless authentication

Seamlessly authenticate users.

Frictionless authentication is when you use the `auth.getCanvaUserToken` method to retain a user's data without them having to sign-in via a login screen. With this, you can do away with traditional, manual authentication entirely or delay it until a user tests your app to see the value it provides.

User tokens

Using the `auth.getCanvaUserToken` method you can:

- Retrieve a token that uniquely identifies the Canva user and their associated brand
- Store the user ID on your end
- Learn about how your users use your app.

We've seen that if a user experiences most of an app's features, they're less likely to abandon the flow than if they're required to authenticate up front.

High-level workflow

The workflow looks like the following:

1. The user interacts with your app. Canva assigns a unique user ID to the user.
2. You track how the user uses your app. This interaction, as well as the associated user and app ID, is stored in the app backend.
3. Your app backend stores the user ID in a database.

As well as the above, you, as the app developer exploring metrics, can use the stored data to know more about your users, such as:

- Track usage: You can monitor how much a user has used your services in Canva, regardless of whether they have interacted with the design. This is especially useful for AI apps where there's a real cost to serve for running the model. Here, you do all the tracking using the data in your app's backend.
- Track time in app: You can save the date the user initially created their account (When the user token was created), then compare that date to today's date to see how long they've been using your app.

Prerequisites

Before you begin, review the Authentication design guidelines.

Step 1: Get a JWT for the current user

To implement frictionless authentication, an app needs the ID of the user and their team, which you can get using a JSON Web Token (JWT).

To get a JWT for the current user:

1. Import the `auth` namespace from the `@canva/user` package:

```
import { auth } from "@canva/user";
```

ts

Call the `getCanvaUserToken` method:

```
const token = await auth.getCanvaUserToken();
```

ts

2. This method returns a JWT as a string.

Step 2: Verify the JWT

By itself, the JWT is a meaningless string of characters. To get the ID of the user and their team from the JWT, an app must send the JWT to the app's backend then verify that JWT.

To send the JWT to the app's backend, use the Fetch API, or a library such as axios:

```
const response = await fetch("http://localhost:3001/my/api/endpoint", {
  headers: {
    Authorization: `Bearer ${token}`,
  },
});
```

ts

When sending the request, include an `Authorization` header that contains the word `Bearer` and the JWT, separated by a space.

When the backend receives the request, extract the JWT from the `Authorization` header. The following code demonstrates how to extract a JWT from an Express.js `Request` object:

```
import express from "express";

const app = express();

app.post("/my/api/endpoint", async (request, response) => {
  const token = getTokenFromHeader(request);

  if (!token) {
    return response.sendStatus(401);
  }

  return response.sendStatus(200);
});

app.listen(process.env.PORT || 3000);

function getTokenFromHeader(request: express.Request) {
  const header = request.headers["authorization"];

  if (!header) {
    return;
  }

  const parts = header.split(" ");

  if (parts.length !== 2 || parts[0].toLowerCase() !== "bearer") {
    return;
  }

  const [, token] = parts;

  return token;
}
```

ts

To verify the JWT, follow the steps in Verifying JWTs. If the JWT is valid, you will end up with an object that contains the ID of the user and their team.

Step 3: Associate data with the user

When the user opens the app for the first time, use the ID of the user and their team to create a user record in the backend's database. This is essentially a registration process, except that it's invisible to the user — an account is created for them in the app's backend when all they've done is access the app.

As the user interacts with the app, or when they return to the app at a later time, send additional HTTP requests to create, read, or update data associated with their account. With each request, the app will need to send and verify a JWT in order to get the ID of the user and their team.

API reference

- `auth.getCanvaUserToken`
- `auth.requestAuthentication`

Manual authentication

How to set up authentication with a third-party platform.

Overview

Manual authentication is the more traditional approach when a user logs in via a third-party so they can access certain content or features within the app.

By doing this, you can:

- Give users access to their personal content, such as photos or videos
- Give users access to certain features based on their specific privileges
- Monetize apps by offering features or content to paying customers.

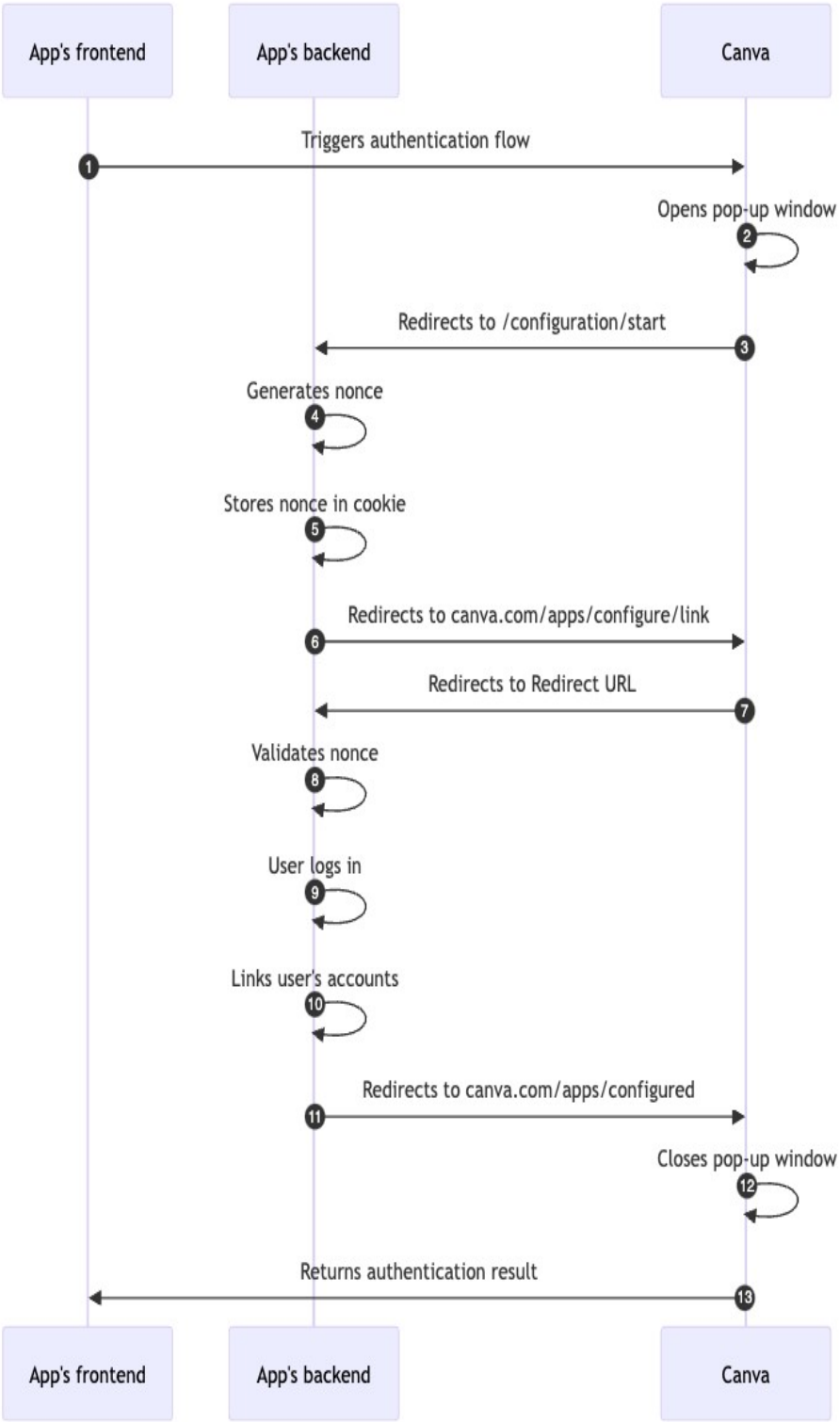
Some examples of apps that support authentication include:

- D-ID Ai Presenters
- Soundraw

For more examples, see the Apps Marketplace.

Lifecycle

The diagram below illustrates the lifecycle of an authentication flow that integrates with a third-party platform. Refer to the instructions below to understand how to implement each of the steps.



Prerequisites

Before you begin, review the Authentication design guidelines.

Step 1: Enable authentication

By default, authentication is disabled.

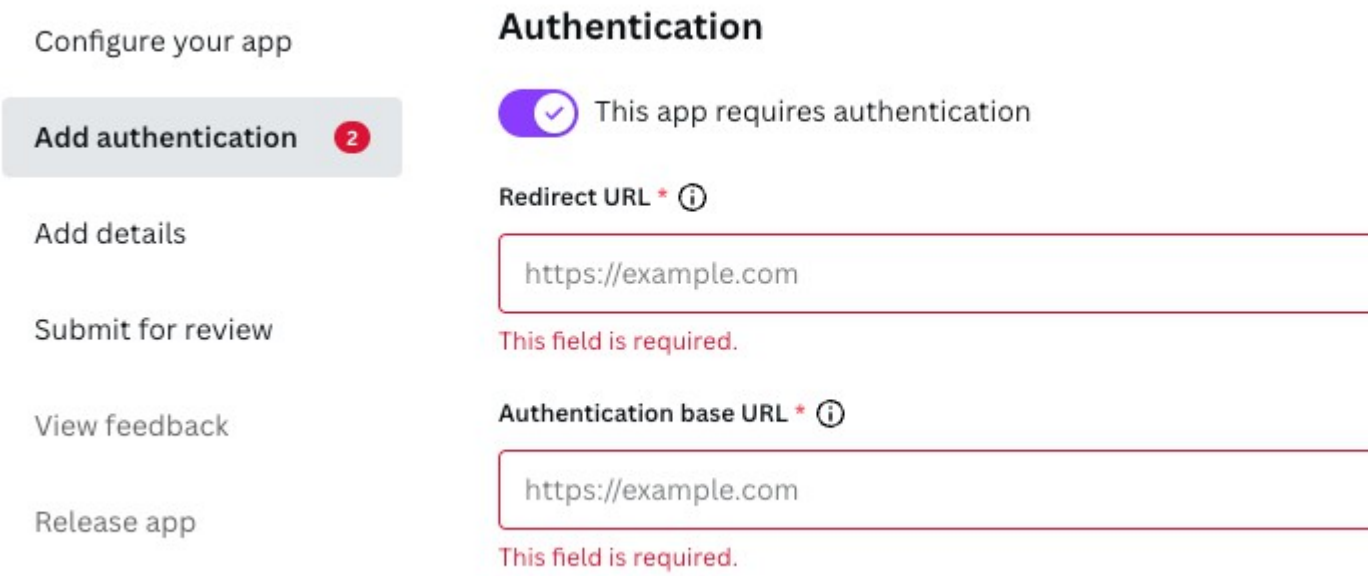
To enable authentication:

1. Log in to the Developer Portal.
2. Navigate to an app via the Your apps page.
3. Click **Add authentication**.
4. Enable **This app requires authentication**.

When authentication is enabled, the following fields become active:

- Redirect URL
- Authentication base URL

Both of these fields must contain a URL and, in the coming steps, we'll set up those URLs.



Step 2: Start an authentication flow

Apps are responsible for triggering the start of an authentication flow. *When* an app triggers the start of an authentication flow depends on the desired user experience. For example, you might want to start an authentication flow when a user tries to access restricted content.

To trigger the start of an authentication flow:

1. Import the `auth` namespace from the `@canva/user` package:

```
import { auth } from "@canva/user";
```

ts

- Call the `requestAuthentication` method:

```
const result = await auth.requestAuthentication();
```

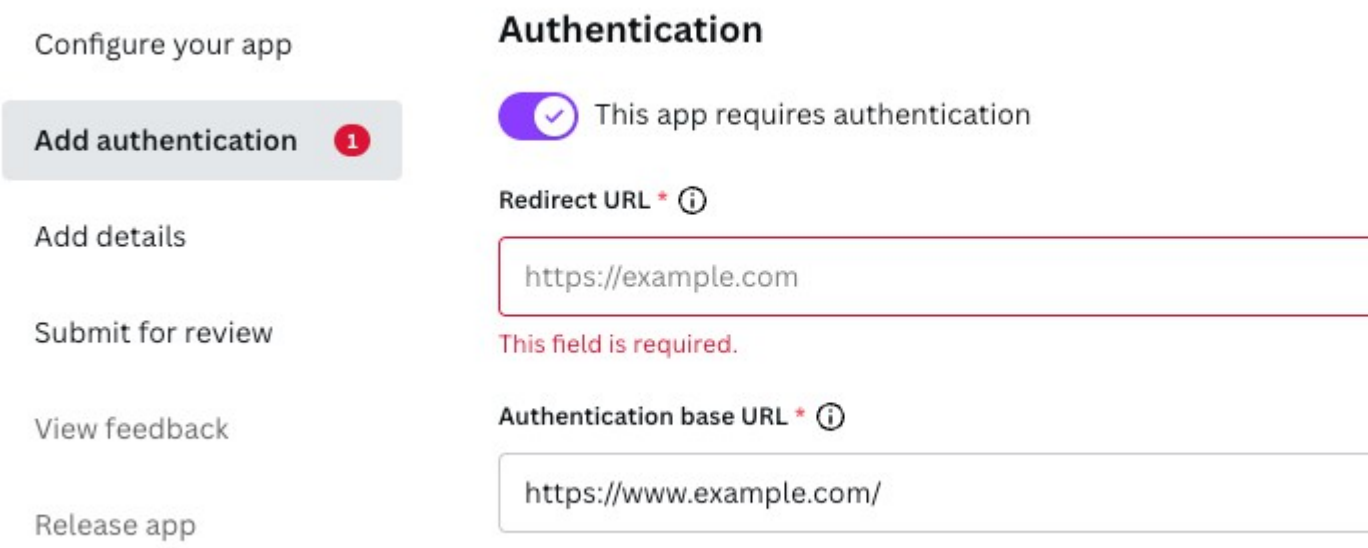
ts

The `requestAuthentication` method opens a popup window and, within this window, redirects the user to the following URL:

```
<authentication_base_url>/configuration/start
```

`<authentication_base_url>` is a placeholder that will be replaced with the app's

Authentication base URL. You can configure this URL in the Developer Portal, via the **Add authentication** page.



For example, if the **Authentication base URL** is:

```
https://www.example.com
```

Then the complete URL would be:

```
https://www.example.com/configuration/start
```

This URL doesn't exist yet though, so we'll set it up in the next step.

Exposing the backend

The **Authentication base URL** must be available to Canva's backend. This means it must be exposed via the public internet. If it's not, Canva won't be able to send requests to it.

To do this, we recommend either of the following options:

- Deploy the app's backend to a non-production environment.
- Use an SSH tunneling service, such as ngrok, to expose a local server.

There are inherent risks in using tools such as ngrok. Be careful about what you expose to the public internet and shut-down the ngrok tunnel as soon as you're done using it.

Step 3: Set up the `/configuration/start` endpoint

In the app's backend, set up an endpoint that handle `GET` requests sent to `/configuration/start`:

```
import * as express from "express";
import * as crypto from "crypto";

const app = express();

app.get("/configuration/start", (req, res) => {
  // TODO: Handle the request
});
```

```
// Start the server
const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

ts

This endpoint must:

- Extract the `state` parameter from the request's query parameters
- Generate a unique nonce for each request
- Store the nonce and an expiry time in a cookie
- Sign the cookie to avoid cookie tampering
- Redirect the user back to Canva

Don't attempt to verify the HTTP request that's sent to this endpoint. It does not have access to the necessary parameters that make verification possible.

Extracting the `state` parameter

Canva appends a `state` query parameter to the `/configuration/start` endpoint. This parameter contains a random string of characters that must be returned to Canva for security reasons.

For the time being, extract the parameter from the request object:

```
const { state } = req.query;
```

ts

Generating a nonce

In the `/configuration/start` endpoint, generate a unique *nonce* for each request.

A nonce is a random, single-use value that's impossible to guess or enumerate. We recommended using a Version 4 UUID that is cryptographically secure, such as one generated with `randomUUID`:

```
import * as crypto from "crypto";
const nonce = crypto.randomUUID();
```

ts

Not all UUIDs are cryptographically secure, such as ones generated with the `uuid` package.

Setting a cookie

After generating a nonce, set a cookie that contains the nonce and an expiry time.

The cookie should be:

- Secure
- HTTP-only
- Signed

It should also have an expiry time **in addition to** the expiry time being included in the cookie itself.

By *signing* a cookie, the app's backend can verify that the user hasn't modified the cookie. This prevents bad actors from tampering with the cookie in the browser.

The steps for setting and signing cookies depend on the framework. Some frameworks automatically sign cookies, while some don't have built-in support for cookie signing and require custom code.

It's beyond the scope of this documentation to explain all of the possible approaches, but refer to the following links to learn more about cookie signing in some popular frameworks:

- Go
  - Fiber
- Node.js
  - Express.js
  - Remix
- PHP
  - Laravel
- Python
  - Django
  - Flask
- Ruby
  - Ruby on Rails

The following code sample demonstrates how to set cookies with Express.js and `cookieParser`:

```
import * as express from "express";
import * as crypto from "crypto";
import * as cookieParser from "cookie-parser";

const app = express();

// TODO: Load a cryptographically secure secret from an environment variable
app.use(cookieParser("SECRET GOES HERE"));

// The expiry time for the nonce, in milliseconds
const NONCE_EXPIRY_MS = 5 * 60 * 1000; // 5 minutes

app.get("/configuration/start", (req, res) => {
  // Generate a nonce
  const nonce = crypto.randomUUID();

  // Create an expiry time for the nonce
  const nonceExpiry = Date.now() + NONCE_EXPIRY_MS;

  // Store the nonce and expiry time in a stringified JSON array
  const nonceWithExpiry = JSON.stringify([nonce, nonceExpiry]);

  // Store the nonce and expiry time in a cookie
  res.cookie("nonceWithExpiry", nonceWithExpiry, {
    httpOnly: true,
    secure: true,
    signed: true,
    maxAge: NONCE_EXPIRY_MS,
  });
});
```

ts

It's worth nothing that:

- To sign a cookie, you need a cryptographically secure secret, such as one generated by the `randomUUID` method. This secret should:
  - be loaded via an environment variable
  - not change between server restarts
  - not be committed to source control
- To store the nonce and an expiry time in a cookie, the above code sample stringifies a JSON array that contains both of the values, but there are other ways to accomplish the same outcome.
- You should be mindful of framework-specific nuances. For example, in Express.js, the `maxAge` property is set in milliseconds, but the `Max-Age` attribute value must be set in seconds.

Redirecting back to Canva

After setting the cookie, redirect the user to the following URL with a `302` redirect:

```
https://www.canva.com/apps/configure/link?state=<state>&nonce=<nonce>
```

You'll need to:

- Replace `<state>` with the `state` query parameter that Canva included with the request.
- Replace `<nonce>` with the nonce that was stored in the cookie.

For example:

```
// Extract state from query parameters
const { state } = req.query;

// Create query parameters
const params = new URLSearchParams({
  state,
  nonce,
});

// Redirect the user
res.redirect(
  302,
  `https://www.canva.com/apps/configure/link?${params.toString()}`
);
```

ts

When the redirect completes, Canva redirects the user to the app's **Redirect URL**. We'll see how to set up the **Redirect URL** in the next step.

Step 4: Set up a Redirect URL

To authenticate users, apps need a **Redirect URL** — a page, hosted on the app's infrastructure, that allows a user to authenticate with the app's platform.

You can configure the app's **Redirect URL** in the Developer Portal, via the **Add authentication** page.

How a user authenticates isn't strictly important. The **Redirect URL** could point to a login form with a username and password, an OAuth 2.0 authorization flow, or something else altogether. The key is that the platform can identify the user and then link that identity to the user's Canva account.

When Canva redirects to the **Redirect URL**, it appends the following query parameters to the URL:

- `canva_user_token`
- `nonce`
- `state`

Your app's backend can use these parameters to securely authenticate a user.

You may notice some other parameters appended to the **Redirect URL**. These parameters are deprecated and should be ignored.

Validating the nonce

When a user arrives at the **Redirect URL**, the app needs to validate the nonce that was generated in the `/configuration/start` endpoint. This ensures that the user who started the authentication flow is the same one who is continuing it (and not some attacker trying to impersonate the user).

To validate the nonce:

1. Get the `nonce` query parameter:

```
const nonceQuery = req.query.nonce;
```

ts

- Get the cookie that contains the nonce and its expiry time:

```
const nonceWithExpiryCookie = req.signedCookies.nonceWithExpiry;
```

ts

- Clear the cookie:

```
res.clearCookie("nonceWithExpiry");
```

ts

Parse the value of the cookie:

```
try {
  const [nonceCookie, nonceExpiry] = JSON.parse(nonceWithExpiryCookie);
} catch (e) {
  // TODO: Handle errors
}
```

ts

- (This code uses a try/catch block to account for the possibility of JSON parsing exceptions.)
- Verify that:
- The nonce from the cookie has not expired.
- The nonces are not nullish.
- The nonces are not empty strings.
- The nonces are equal to one another.

For example:

```
if (
  Date.now() > nonceExpiry || // The nonce has expired
  typeof nonceCookie !== "string" || // The nonce in the cookie is not a string
  typeof nonceQuery !== "string" || // The nonce in the query parameter is not a
string
  nonceCookie.length < 1 || // The nonce in the cookie is an empty string
  nonceQuery.length < 1 || // The nonce in the query parameter is an empty
string
```

```
  nonceCookie !== nonceQuery // The nonce in the cookie does not match the nonce
in the query parameter
) {
  // The nonce is NOT valid
}
```

ts

Handling invalid nonces

If the nonce validation fails, redirect the user to the following URL with a `302` redirect:

```
https://www.canva.com/apps/configured?
success=false&state=<state>&errors=<errors>
```

You'll need to:

- Replace `<state>` with the `state` query parameter that Canva included with the request.
- Replace `<errors>` with a comma-separated list of one or more error codes. You can define the error codes yourself. They will be passed as-is to the app's frontend.

For example:

```
// Get the nonce from the query parameter
const nonceQuery = req.query.nonce;

// Get the nonce with expiry time from the cookie
const nonceWithExpiryCookie = req.signedCookies.nonceWithExpiry;

try {
  // Parse the JSON that contains the nonce and expiry time
  const nonceWithExpiry = JSON.parse(nonceWithExpiryCookie);

  // Extract the nonce and expiry time
  const [nonceCookie, nonceExpiry] = nonceWithExpiry;

  // Clear the cookie
  res.clearCookie("nonceWithExpiry");

  // If the nonces are invalid, terminate the authentication flow
  if (
    Date.now() > nonceExpiry || // The nonce has expired
    typeof nonceCookie !== "string" || // The nonce in the cookie is not a
string
    typeof nonceQuery !== "string" || // The nonce in the query parameter is not
a string
    nonceCookie.length < 1 || // The nonce in the cookie is an empty string
    nonceQuery.length < 1 || // The nonce in the query parameter is an empty
string
    nonceCookie !== nonceQuery // The nonce in the cookie does not match the
nonce in the query parameter
  ) {
    const params = new URLSearchParams({
      success: "false",
      state: req.query.state,
      errors: "invalid_nonce",
    });

    return res.redirect(
      302,
      `https://www.canva.com/apps/configured?${params.toString()}`
    );
  }
```

```
} catch (e) {
  // An unexpected error has occurred (e.g. JSON parsing error)
  const params = new URLSearchParams({
    success: "false",
    state: req.query.state,
    errors: "invalid_nonce",
  });

  return res.redirect(
    302,
    `https://www.canva.com/apps/configured?${params.toString()}`
  );
}
```

ts

We also recommend logging a security alert, as invalid nonces suggest a potential threat.

Authenticating the user

If the nonce validation succeeds, allow the user to authenticate. This is the part of the process that is highly dependent on the third-party platform, as there are many authentication strategies available.

After the user authenticates — for example, by entering a username and password — the next step is to create a link between the user's account on the platform and the user's account with Canva.

To link a user's accounts:

1. Grab the `canva_user_token` query parameter that Canva includes with the request. This parameter is a JSON Web Token (JWT) that is unique to the user.
2. Verify the JWT to get the ID of the user and their team.
3. Use the IDs to create a mapping between the user in Canva and the user in the app's backend.

There are many ways to create a mapping, so it's not practical to document all of the possible options, but we can provide an example:

If your app's backend has a `users` table in its database, you could add an `canvaId` column to the table. This column could contain a composite key made up of the IDs. For example, if the user's ID is `123` and the ID of their team is `456`, the composite key would be something like `123:456`.

When a user authenticates via the **Redirect URL**, the app could save the composite key to the database. The app could then check for the existence of this key to determine if the user is authenticated.

Step 5: End the authentication flow

At the end of an authentication flow, the app needs to:

- Know that the authentication flow has ended.
- Know the outcome of the authentication flow.
- Close the popup window.

If the user is able to successfully authenticate with the third-party platform, redirect them to the following URL from within the popup window:

```
https://www.canva.com/apps/configured?success=true&state=<state>
```

Replace `<state>` with the value of the `state` token from the start of the authentication flow.

If the user is *not* able to authenticate — for example, they have too many failed login attempts — redirect them to the following URL:

```
https://www.canva.com/apps/configured?
success=false&state=<state>&errors=<errors>
```

Be sure to:

- Replace `<state>` with the value of the `state` token from the start of the authentication flow.
- Replace `<errors>` with a comma-separated list of one or more error codes. You can define the error codes yourself. They will be passed as-is to the app's frontend.

If you're performing the redirect via the app's backend, use a `302` redirect:

```
const params = new URLSearchParams({
  success: "true",
  state: req.query.state,
});

response.redirect(
  302,
  `https://www.canva.com/apps/configured?${params.toString()}`
);
```

ts

If you're performing the redirect via the app's frontend, use the `window.location.replace` method:

```
const params = new URLSearchParams({
  success: "true",
  state: req.query.state,
});

window.location.replace(
  `https://www.canva.com/apps/configured?${params.toString()}`
);
```

ts

Step 6: Handle the authentication result

When the authentication flow ends, successfully or otherwise, the `requestAuthentication` method returns an object with a `status` property that describes the outcome of the flow:

```
const result = await auth.requestAuthentication();
console.log(result.status);
```

ts

You can use the `status` property to render an appropriate user interface based on the result of the flow — for example, rendering an error when the authentication fails, or rendering certain content or features when authentication succeeds.

The possible values of the `status` property are:

- `"COMPLETED"` - The user was able to authenticate.
- `"DENIED"` - The user was not able to authenticate.
- `"ABORTED"` - The user closed the popup window.

When the `status` property is `"DENIED"`, the object has a `details` property that contains an array of error codes. These are the error codes provided by the app (if any) when it redirected back to Canva.

Step 7: Send authenticated requests

When a user opens or interacts with an app, the app will need to check if the user is authenticated and, in some cases, render privileged content or features for that user.

To accomplish this, generate a JWT for the current user in the app's frontend:

```
const token = await auth.getCanvaUserToken();
```

ts

Then send an HTTP request to the app's backend with the JWT in the `Authorization` header:

```
const response = await fetch("https://localhost:3001/authenticated-endpoint", {
  method: "POST",
  headers: {
    Authorization: `Bearer ${token}`,
  },
});
```

ts

When the backend receives the request:

1. Extract the JWT from the `Authorization` header.
2. Verify the JWT to get the ID of the user and their team.
3. Use the IDs to check if the user is authenticated. How the app does this depends on how the app created a mapping between the user in their backend and the user on Canva.

If the user is authenticated, respond with whatever the frontend needs to render the authenticated content or features — for example, the user's photos or specific privileges.

If the user is not authenticated, respond to the request with an error. The frontend can use this error to restrict what the user has access to and to prompt them to authenticate.

Step 8: Handle disconnections

After a user connects (installs) an app, they have the option of disconnecting (uninstalling) it.

If a user has authenticated with a third-party platform, disconnecting the app in Canva won't automatically remove the connection between Canva and the platform — as far as the platform is concerned, the user will still be authenticated.

This is a subpar user experience for a couple of reasons:

- Disconnecting an app signals that the user's intent is to no longer be associated with the app and the app should respect that intent.
- If the user reconnects the app at a later time, they'll already be authenticated. This is a confusing user experience that's inconsistent with how the the rest of Canva works.

To address this issue, Canva sends a `POST` to the following endpoint when a user disconnects an app:

```
<authentication_base_url>/configuration/delete
```

`<authentication_base_url>` is a placeholder that will be replaced with the app's

Authentication base URL. You can configure this URL in the Developer Portal, via the **Add authentication** page.

The `/configuration/delete` endpoint must not redirect to a different endpoint. Even a redirect that just appends a `/` to the end of the endpoint will cause the request to fail.

In the request's headers, Canva includes an `Authorization` header that contains a JWT for the current user. When the app receives this request, it should:

1. Extract the JWT from the `Authorization` header.
2. Verify the JWT to get the ID of the user and their team.
3. Use the IDs to find the user in the app's backend.
4. Remove any connection between the user and Canva.

When the disconnection is complete, respond with a `200` status code and the following object:

```
{
  "type": "SUCCESS"
}
```

json

To confirm that the disconnection flow is working:

1. Navigate through the authentication flow.
2. Disconnect (uninstall) the app.
3. Reconnect (install) the app.

You should have to re-authenticate to access any privileged content or features. If you don't have to re-authenticate, double-check the disconnection logic.

API reference

- `auth.getCanvaUserToken`
- `auth.requestAuthentication`